Pipeline and Reporting Mart

Pipeline and Reporting Mart Low-Level Design

1. Document Control

Item	Value
System	ITIL Group Reports Automation
Component	Pipeline, Transform Rules, and Reporting Mart
Owner	CNE Automation / ITIL Reporting
Status	QA implementation active

2. Purpose

The pipeline extracts operational records from Helix QA/API or compatible files, transforms them into report-ready records, loads staging tables, and runs dbt to generate the daily report mart.

The automated wrapper is scheduled for 07:00 every day, including weekends. This creates consecutive 24-hour successful-closure windows. Automated report emails remain weekday-only. Direct manual pipeline commands remain available for controlled recovery runs.

3. Scope

In scope:

- Helix and file extraction.
- Transform rules for division, category, SLA status, hours open, and report inclusion.
- Staging load behavior.
- dbt staging and mart generation.
- Scheduler, heartbeat, run log, and operational support checks.

Out of scope:

- User auth and role implementation. See `docs/auth-access-lld.md`.
- Division admin API behavior. See `docs/division-management-lld.md`.
- Daily report API payload design. See `docs/daily-report-lld.md`.

4. Runtime Components

Component	Path	Responsibility
Pipeline orchestrator	`pipeline/run.py`	Runs extract, transform, load, dbt, heartbeat
Helix extractor	`helix/extractor.py`	Pulls records from Helix API
File extractor	`pipeline/extract.py`	Reads local Excel/CSV drops
Transformer	`pipeline/transform.py`	Applies business rules
Loader	`pipeline/load.py`	Upserts staging records and writes run metrics
Division mapping service	`pipeline/division_groups.py`	Supplies classification mappings
dbt staging model	`dbt/models/staging/stg_changes.sql`	Deduplicates and normalizes staging data
dbt summary mart model	`dbt/models/marts/daily_report_summary.sql`	Builds report matrix
dbt records mart model	`dbt/models/marts/daily_report_records.sql`	Builds record-level daily snapshots
Health API	`api/routers/health.py`	Exposes health, heartbeat, and run-log status

5. Overall Connection Diagram

```
Helix QA API / Excel or CSV drops
        |
```

```
        v
Extraction
  - helix/extractor.py
  - pipeline/extract.py
        |
        v
Canonical records
  - pipeline.models.ChangeRecord
        |
        v
Transform rules
  - division from config.division_groups
  - category from op_cat_tier1 or request prefix
  - SLA status from schedule/assignment timestamps
  - inclusion/exclusion flags
        |
        v
Staging load
  - staging.raw_changes
  - staging.elt_run_log
        |
        v
dbt
  - dbt.stg_changes
  - reporting.daily_report_summary
  - reporting.daily_report_records
        |
        v
API consumers
  - /api/v1/daily-reports
  - /api/v1/daily-reports/records
  - /api/v1/operational-records
  - /api/v1/health
  - /api/v1/pipeline-runs
        |
        v
Frontend
  - daily report pages
  - operations console
  - admin/status screens

```

The pipeline owns record classification and SLA status. The API and frontend consume the derived fields; they must not recalculate SLA independently.

6. Scheduled Pipeline Flow

1. Linux cron or an operator starts `deploy/run-pipeline.sh`.
2. The script defaults to `PIPELINE_SOURCE=helix`.
3. `pipeline/run.py` opens a row in `staging.elt_run_log`.
4. The Helix extractor pulls records for the configured period, or file extraction reads pending Excel/CSV drops.
5. Records are transformed in memory by `pipeline.transform`.
6. Transformed records are loaded into `staging.raw_changes`.
7. dbt runs against the project in `dbt/`.

8. dbt creates or refreshes `reporting.daily_report_summary` and `reporting.daily_report_records`. It also creates `reporting.daily_report_success_closures`, containing the per-group count fetched by the morning run for records whose Actual End Date was yesterday and whose Status Reason was Success.
9. `reporting.pipeline_heartbeat` is updated.
10. The API serves latest staging and mart data.
11. The unattended email scheduler evaluates enabled division/group subscriptions after a fresh successful run.

Manual report delivery is also available to Admin and Super Admin users. It uses the same snapshot tables and attachment generator but ignores the automatic schedule. Automatic and manual sending are independently disabled by default through `REPORT_EMAIL_AUTOMATION_APPROVED=false` and `REPORT_EMAIL_MANUAL_SEND_ENABLED=false`.

Email delivery is executed by the backend so scheduled sends remain unattended. The HTML renderer follows the frontend-approved Vodacom visual specification using Outlook-compatible table markup and inline styles. CSV/XLSX attachments mirror the Daily Report investigation export contract in the frontend: SLA, Request ID, Task ID, Category, Priority, Group, Assignee, Hours Open, Scheduled End, and Summary. Neither the API nor the frontend independently recalculates the snapshot metrics used by the email.

7. Data Model

7.1 `staging.raw_changes`

Stores transformed operational records from Helix or file ingestion.

Column	Purpose
`request_id`	Root ticket/request ID
`task_id`	Task ID
`assignee_group`	Source assignee group
`report_group`	Derived division
`report_category`	Derived report category
`sla_status`	Derived SLA status

`hours_open`	Hours since assignment or until close
`exclude_change`	Exclusion flag for change records
`exclude_incident`	Exclusion flag for non-change records
`source`	`helix_api`, `excel`, or compatible source
`run_id`	Link to `staging.elt_run_log`
`loaded_at`	Last load timestamp

Conflict key:

```
(request_id, task_id)
```

7.2 `dbt.stg_changes`

dbt view over `staging.raw_changes`.

Responsibilities:

- Deduplicate by `(request_id, task_id)`.
- Keep the latest `loaded_at`.
- Trim common string fields.
- Preserve classification fields for mart generation.

7.3 `reporting.daily_report_summary`

dbt table used by the Daily Report API and matrix consumers.

Grain:

```
report_date + report_group + assignee_group + sla_band
```

Measures:

- `incident_count`
- `change_count`
- `access_count`
- `event_count`

- `total_count`

7.4 `reporting.daily_report_records`

dbt table used by Daily Report drill-through and selected-date group metadata.

Grain:

```
report_date + request_id + task_id
```

Purpose:

- Preserve the eligible record set for each report date.
- Let a selected `report_date` drive KPIs, matrix rows, groups, and records together.
- Prevent historical report screens from mixing yesterday's KPI totals with today's current staging records.

The models are incremental and replace only the current Johannesburg-date snapshot during a dbt run. A snapshot includes only staging rows whose `loaded_at` falls on that same `WORKSPACE_TIMEZONE` date. Older rows are never mixed into today's snapshot, and older `report_date` snapshots remain preserved.

8. Business Rules

8.1 Division Classification

Division is derived by matching `assignee_group` against active patterns from `config.division_groups`.

Ordering:

1. Database-managed `config.division_groups`.
2. Static fallback mappings, only if DB mappings cannot be loaded.
3. Longer patterns first so more specific mappings win.

Unmatched records are not included in Daily Report totals because they do not have a report division.

8.2 Report Category Classification

Report category starts from a dashboard-compatible task type derivation.

If `OpCatTier1` is present after trimming:

`OpCatTier1` value	Derived task type
`Failure`	`Alarm`

Any other nonblank value	The trimmed source value

If `OpCatTier1` is blank, task type is inferred from `RootRequestID`, `Summary`, and `Notes`:

Source condition	Derived task type
`RootRequestID` starts with `INC` and `Summary` starts with `ARM:`	`Security`
`RootRequestID` starts with `INC` and `Summary` or `Notes` contains `PASSWORD` or `ACCESS`	`Security`
`RootRequestID` starts with `INC` and `Notes` contains `ARM:`	`Security`
`RootRequestID` starts with `INC` and `Summary` starts with `ALARM` or `Notes` contains `ALARM`	`Alarm`
`RootRequestID` starts with `INC` and none of the above matches	`Faults`
`RootRequestID` starts with `CRQ`	`Change`
`RootRequestID` starts with `WO`	`Work Order`
`RootRequestID` starts with `PBI`	`PBI`
Any other blank-category record	`Alarm`

Derived task type is then mapped to report category:

Derived task type	Report category
`Faults`	`INCIDENT`
`Change`	`CHANGE`
`Security` or `Access`	`ACCESS`
`Alarm` or `Failure`	`EVENT`
`Work Order` or `PBI`	Excluded from report totals

8.3 SLA Status Calculation

SLA status is calculated in `pipeline/transform.py` by `_sla_status(rec, now)`. The source of truth for thresholds is `config/mappings.py`.

Input fields:

Field	Used for
`status`	Determines whether the record is open, closed, or out of scope
`report_category`	Selects the change SLA path or assignment-time SLA path
`assign_time`	SLA clock start for `INCIDENT`, `ACCESS`, and `EVENT`
`scheduled_end`	SLA due timestamp for `CHANGE`
`closed_time`	Used only for `hours_open` on closed records

Open statuses:

```
Assigned
Staged
Work In Progress
Work in Progress
Pending

```

Closed statuses:

```
Closed
Cancelled
Success
Bypassed

```

Reportable SLA statuses:

Status	Meaning
`WITHIN_SLA`	Open record is not near breach
`NEARING_SLA`	Open record is inside the configured nearing threshold

`BREACHED`	Open record has passed scheduled end or breach threshold
`NO_SCHEDULE`	Open Change has no usable scheduled end date
`NO_ASSIGN_TIME`	Open Incident, Access, or Event has no assign time, so its SLA clock cannot be calculated

Non-open statuses become `CLOSED` or `OUT_OF_SCOPE` and are not included in the daily report matrix.

Change SLA Path

For `report_category = CHANGE`, the calculation is based on `scheduled_end`.

Configured thresholds:

Constant	Current value	Meaning
`CHANGE_SCHEDULE_GRACE_SECONDS`	`1_800` seconds / 30 minutes	Grace period after scheduled end before breach
`CHANGE_NEARING_WINDOW_SECONDS`	`7_200` seconds / 2 hours	Nearing window before the grace-adjusted due time

Formula:

```
due_at = scheduled_end + CHANGE_SCHEDULE_GRACE_SECONDS
near_at = due_at - CHANGE_NEARING_WINDOW_SECONDS

if scheduled_end is missing:
    sla_status = NO_SCHEDULE
else if now > due_at:
    sla_status = BREACHED
else if now > near_at:
    sla_status = NEARING_SLA
else:
    sla_status = WITHIN_SLA

```

Example:

```
scheduled_end = 2026-07-14 10:00
due_at        = 2026-07-14 10:30
near_at       = 2026-07-14 08:30
```

```
now <= 08:30  -> WITHIN_SLA
now > 08:30   -> NEARING_SLA
now > 10:30   -> BREACHED
```

Incident, Access, and Event SLA Path

For `report_category IN (INCIDENT, ACCESS, EVENT)`, the calculation is based on `assign_time`.

Configured thresholds:

Constant	Current value	Meaning
`ASSIGN_TIME_NEARING_SECONDS`	`79_200` seconds / 22 hours	Nearing SLA starts 22 hours after assignment
`ASSIGN_TIME_BREACH_SECONDS`	`86_400` seconds / 24 hours	Breach starts 24 hours after assignment

Formula:

```
near_at = assign_time + ASSIGN_TIME_NEARING_SECONDS
due_at = assign_time + ASSIGN_TIME_BREACH_SECONDS

if assign_time is missing:
    sla_status = NO_ASSIGN_TIME
else if now > due_at:
    sla_status = BREACHED
else if now >= near_at:
    sla_status = NEARING_SLA
else:
    sla_status = WITHIN_SLA
```

Example:

```
assign_time = 2026-07-14 08:00
near_at     = 2026-07-15 06:00
due_at      = 2026-07-15 08:00

now < 06:00  -> WITHIN_SLA
now >= 06:00 -> NEARING_SLA
now > 08:00  -> BREACHED
```

8.4 Hours Open Calculation

`hours_open` is calculated in `pipeline/transform.py` by `_hours_open(rec, now)`.

Formula:

```
if assign_time is missing:
    hours_open = null
else if status is closed and closed_time exists:
    hours_open = (closed_time - assign_time) / 3600
```

```
else:
    hours_open = (now - assign_time) / 3600

```

`hours_open` is a diagnostic and drill-through value. Daily report SLA bands are driven by `sla_status`, not by recalculating from `hours_open`.

In XLSX and CSV attachments, `hours_open` is normalized to a finite numeric value rounded to two decimal places. XLSX cells use the `0.00` number format so teams can sort and filter the column numerically. Missing, invalid, `NaN`, or infinite values are exported as blank cells.

8.5 Mart SLA Bands

`reporting.daily_report_summary` converts record-level `sla_status` into report rows. `reporting.daily_report_records` preserves the same eligible records at record level for drill-through.

Record `sla_status`	Mart `sla_band`
`NEARING_SLA`	`Calls Nearing SLA`
`BREACHED`	`Breached Calls`
`WITHIN_SLA`	Included in `Total Open Calls`
`NO_SCHEDULE`	Included in `Total Open Calls`
`NO_ASSIGN_TIME`	Included in `Total Open Calls`

The mart always creates three bands for each active division/group combination:

```
Total Open Calls
Calls Nearing SLA
Breached Calls

```

`Total Open Calls` is a full open-record count. It is not only within-SLA records. This means:

```
Total Open Calls >= Calls Nearing SLA
Total Open Calls >= Breached Calls

```

8.6 Inclusion and Exclusion

Records are eligible for daily reports only when all conditions are true:

- Record has an active mapped division in `report_group`.
- `report_category` is `INCIDENT`, `CHANGE`, `ACCESS`, or `EVENT`.
- Status is one of the configured open statuses.

- Assignee is not in `EXCLUDED_ASSIGNEES`.
- Operational category and request prefix do not match the Work Order/PBI exclusions, except that a Security Work Order classified as `ACCESS` remains eligible.

Missing schedule or assignment timestamps do not make an otherwise eligible record disappear. They produce `NO_SCHEDULE` for Changes or `NO_ASSIGN_TIME` for Incident, Access, and Event tasks.

9. Monitoring and Observability

`staging.elt_run_log` records source, status, timing, rows extracted, rows loaded, rows rejected, and error text. `reporting.pipeline_heartbeat` stores latest pipeline status for health checks and operator dashboards.

Pipeline logs must make source selection, extraction count, transform count, SLA distribution, rejected count, dbt result, heartbeat status, and run close status visible.

10. Error Handling

Failure	Handling
Helix API failure	Mark run failed, write error, keep prior mart data
File parse failure	Reject bad rows or fail the file depending on parser stage
DB unavailable during load	Mark run failed when possible and surface DB error
dbt failure	Mark run failed after load; do not claim mart refresh success
Watermark/schema drift	Use defensive fallback and log exact issue
Unknown division/group	Add or fix mapping and rerun pipeline
Missing Change scheduled end	Classify as `NO_SCHEDULE` if otherwise reportable
Missing Incident/Access/Event assign time	Classify as `NO_ASSIGN_TIME` if otherwise reportable

11. Testing Strategy

Unit-level checks:

- Division group matching.
- Category classification.
- Change SLA status: missing schedule, within, nearing, breached.
- Incident/access/event SLA status: missing assignment, within, nearing, breached.
- Hours-open calculations.
- Inclusion/exclusion rules.
- Mart banding: total open includes within, nearing, breached, and no-schedule records.

Pipeline smoke tests:

```
python pipeline/run.py --source helix
python pipeline/run.py --source files
dbt compile
dbt run

```

Expected checks:

- `staging.elt_run_log` records the run.
- `staging.raw_changes` contains loaded rows.
- `reporting.daily_report_summary` contains report rows.
- `reporting.daily_report_records` contains selected-date drill-through rows.
- `reporting.pipeline_heartbeat` reflects latest status.
- Category totals sum to `total_count`.
- `Total Open Calls` is greater than or equal to the nearing and breached bands for each division/group.

12. Support Runbook

Pipeline run failed:

1. Check latest `staging.elt_run_log`.
2. Check application/pipeline logs for extract, load, or dbt phase.
3. Confirm DB connectivity.
4. Confirm Helix credentials and query period.
5. Rerun after fixing the failed phase.

Report mart is stale:

1. Check `reporting.pipeline_heartbeat`.
2. Confirm the latest run reached dbt.
3. Run dbt manually if load succeeded but mart refresh failed.
4. Confirm `reporting.daily_report_summary` row count for the expected date.
5. Confirm `reporting.daily_report_records` row count for the expected date.

Division is missing after mapping change:

1. Confirm mapping exists in `config.division_groups`.
2. Rerun the pipeline so records are classified with the new mapping.
3. Confirm `staging.raw_changes.report_group`.
4. Confirm mart rows exist for the division.

SLA counts look wrong:

1. Check `staging.raw_changes.sla_status` for the affected records.
2. Confirm `report_category` is correct because changes and incidents use different SLA clocks.
3. For changes, compare `scheduled_end` with the grace-adjusted due time.
4. For incidents/access/events, compare `assign_time` with the 22-hour nearing and 24-hour breach thresholds.
5. Confirm the record is not excluded by category, prefix, unknown division, or non-open status.
6. Confirm dbt refreshed `reporting.daily_report_summary` and `reporting.daily_report_records` after the latest staging load.

13. Acceptance Criteria

Pipeline and Reporting Mart is accepted when Helix or file records can be extracted and transformed, staging load is idempotent on `(request_id, task_id)`, SLA status is calculated from the documented change and incident/access/event rules, dbt produces `reporting.daily_report_summary` and `reporting.daily_report_records`, operational status is visible, and Daily Report APIs can consume fresh data without owning pipeline logic.